HOW MUCH HUMAN WORK IS BEHIND AI-ASSISTED SOFTWARE?
LABOUR BOUNDS AND ESTIMATION FROM WORK TELEMETRY

Abstract. Software-work telemetry records events whose relationship to human time can change under AI assistance. We determine which labour totals, measured in allocated person-hours, are consistent with specified observations. We report elapsed time to acceptance and output retained at follow-up separately. Observed interval totals or justified lower and upper bounds, together with each person's available time, constrain how much labour could have occurred within a target window. Splitting time at every observation endpoint and target-window boundary gives a finite linear programme with the same possible labour totals, even when observations overlap or tasks share a person's time. Minimising and maximising the target total gives its exact bounds; every total between them is possible under the same constraints, even when each person does one activity at a time and may switch activities at arbitrary times. Adding bounds calculated separately for each interval can include totals that no joint allocation can produce. For a probability sample, we add differences between observed and predicted labour, weighted by inverse inclusion probabilities, to the total of predictions fixed before sampling. Under the stated sampling and measurement assumptions, this estimate averages to true labour over repeated samples. Its variance can be higher or lower than that of the estimator without predictions. We supply analytical proofs and exact finite computational checks using constructed data. The proposed measurements still require empirical validation.

1. What the observations determine

Suppose a team records three person-hours of work during a four-hour interval. How much work occurred during its first two hours? If one person did all the work, at least one person-hour must fall in the first half, because at most two can fall in the second. At most two person-hours can fall in the first half. The person could work one hour in the first half and two in the second, or two in the first and one in the second. The recorded total therefore does not tell us exactly how much work occurred in the first two hours. Commit times alone may not even determine total labour: the same event sequence can arise from different amounts of human activity.

Labour is human time allocated to the included work, measured in person-hours. For AI-assisted software work, we count the time spent on all included types of work and report elapsed time to first acceptance and reference output retained at follow-up separately. The accounting includes unsuccessful work and post-acceptance repair. At any time, the fractions of a person's time allocated to different tasks and types of work must sum to at most one. Our linear programme gives the minimum and maximum labour totals consistent with the observed interval totals or justified bounds and each person's available time. Every total between those endpoints is possible under the same constraints. Establishing a higher minimum or a lower maximum therefore requires additional observations or justified restrictions on how people allocated their time.

When exact labour totals are unavailable, a probability sample of person-intervals (time intervals for individual people) can give an estimate that is unbiased over repeated samples, provided inclusion probabilities are known and positive and allocations are measured accurately. For each sampled person-interval, divide observed labour minus predicted labour by the interval's inclusion probability. Add these weighted differences to the total of predictions fixed before sampling. This estimator improves precision when the sum of weighted

Date: 1 November 2026.

Key words and phrases. software engineering, partial identification, labour bounds, activity sampling, model-assisted estimation, AI assistance.

differences has lower variance over repeated samples than the estimator without predictions. Telemetry can therefore improve a labour estimate even when commit events do not uniquely determine labour. Sampling uncertainty describes how an estimate varies with the selected sample. An identification interval instead contains exactly the labour totals consistent with the specified observations and allocation constraints. Estimating how AI changes labour or output additionally requires a causal study design.

Wright and Ziegler's Standard Coder approach [14] infers coding activity from commit timing and predicts the resulting labels from code-change features. Standard Coding Hours (SCH) can serve as a common reference index. Using SCH to measure person-hours requires calibration against independently observed labour. AI may alter both how human activity produces commits and how much work a code change represents. The SPACE framework places activity and efficiency within a broader account of developer productivity [8]. Mozannar et al. examine interaction time costs in AI-assisted programming [11].

Existing AI studies measure different outcomes in different populations. Peng et al. examine completion time among completers of a programming exercise [12]. Becker et al. study experienced maintainers in familiar repositories, comparing log completion times after adjustment for forecast times [2]. Cui et al. study weekly completed pull requests in organisational experiments [6], using allocation to study groups as an instrument for adoption in their June 2025 manuscript [5, Section 4.1 and Table 3]. Their estimates concern different populations, AI tools and working practices, and outcomes, so they cannot be substituted for one another as estimates of the same quantity. METR's subsequent account of selection and concurrent-agent measurement difficulties further motivates separate labour and elapsed-time records [3]. The calculations here use no data from these studies.

Sections 2 and 3 explain why commits alone need not determine person-hours and define labour, acceptance time and retained output. Section 4 derives the minimum and maximum possible labour totals, and Section 5 uses sampled labour to correct fixed telemetry predictions. Section 6 sets out how to validate these measurements before comparing AI tools and working practices. The proofs apply standard optimisation and sampling results to the stated constraints on allocating people's time. Appendix A gives a shortest-path solution for one person's work of one type on one task.

2. Why event telemetry alone need not identify labour

Let a minute-indexed hidden variable $Z_t \in \{0,1\}$ indicate an activity state, and let $C_t \in \{0,1\}$ record a commit event. A fitted event model specifies how activity changes from minute to minute and how activity produces commits. Interpreting $Z_t = 1$ as "coding" requires validation against independently measured labour. Even before that validation, different probability distributions of activity can produce the same distribution of commit events.

Proposition 2.1 (Failure of unrestricted identification)**.** *Suppose the states Z_t are independent Bernoulli variables with parameter $\pi \in (0,1)$, and, independently across minutes and of the states, B_t are Bernoulli variables with parameter $c \in (0,1)$. Define $C_t = Z_t B_t$. The joint distribution of all commit observations depends on (π, c) only through πc. In particular, $(\pi, c) = (1/2, 1/5)$ and $(1/4, 2/5)$ give identical commit distributions but different expected active times.*

Proof. Each C_t is Bernoulli with parameter $\theta = \pi c$, and the C_t are independent. For every $n \geq 1$ and every binary string $(e_1, \ldots, e_n)$,

$$\mathbb{P}(C_1 = e_1, \ldots, C_n = e_n) = \theta^{\sum_t e_t}(1-\theta)^{n-\sum_t e_t}. \tag{1}$$

Both parameter pairs give $\theta = 1/10$. In contrast,

$$\mathbb{E}\left[\sum_{t=1}^{n} Z_t\right] = n\pi, \tag{2}$$

which is $n/2$ minutes in the first model and $n/4$ in the second. Thus identical commit distributions can correspond to different expected numbers of active minutes. □

This construction belongs to the independent-state subfamily of a two-state hidden Markov model: the next activity state has the same distribution for either current state. The example shows that different parameter values in the unrestricted model family can give the same distribution of commits. Models with persistent activity or other restrictions may still be identifiable. Restrictions on the probability model for activity, informative code-change features or independent activity observations may distinguish models that commit timing alone cannot distinguish. A claim that the parameters are identifiable must state which restrictions or observations make that possible. Under AI assistance, commits can continue while human attention is elsewhere, so the mechanism that produces commits may itself change.

Agreement between models does not establish that their predictions agree with independently measured person-hours. Measured interval totals instead constrain how much labour occurred during a specified period.

3. Task accounting with explicit boundaries

3.1. **Tasks, AI tools and person-hours.** Begin tracking task i at time s_i, when its requested behaviour, conditions for inclusion in the study and acceptance criteria are recorded. Identify each task by its requirement, so changing its commits or pull requests does not change which task is counted as output. Before observing outcomes, define how split tasks, merged tasks and scope changes will be handled. When a task is split, the original task can retain its identifier while separate records describe work on its components. Counting both the original task and its components as delivered output would count the same work twice. Keep a history of changes to requirements and acceptance criteria, and state how tasks with those changes will be analysed.

Before comparing AI tools and working practices, specify what counts as one task, whether accepted components count separately or only as part of a completed parent task, and when follow-up starts. Consider separately accepted components regrouped into a parent task that is accepted only after all its components have been accepted. Even when component reference scores add to the parent's score, regrouping can change how much output is counted by a deadline. The comparison therefore depends on how tasks are grouped and accepted as well as how they are scored.

For each comparison group, specify the available AI tools and model versions, the tasks on which they may be used, the actions they may perform autonomously, who reviews their work and the requirements for release. Record actual tool use separately from the tools and working practices specified for that group: difficult tasks may elicit more use, and people may decline to use a tool made available to them.

Let $a_{ijr}(t)$ be the fraction of person j's time spent on task i doing work of type r, such as implementation or review, at calendar time t. With finite sets of tasks, people and types of work, measurable allocation functions define cumulative labour through task age $u \geq 0$, measured from the start of tracking, by

$$a_{ijr}(t) \geq 0, \qquad \sum_{i,r} a_{ijr}(t) \leq 1, \qquad H_i(u) = \sum_{j,r} \int_{s_i}^{s_i+u} a_{ijr}(t)\, dt. \tag{3}$$

With time in hours, $H_i(u)$ has units of person-hours. Include all the work on the task, from specification and implementation through review, testing and repair. Prompting and integration also consume person-hours. Pair work can contribute two person-hours in one elapsed hour. For one person, two concurrent sessions cannot each receive a full hour of attention. State whether attended waiting belongs in the labour total. Record unattended tool execution as computing-resource use.

To measure the task labour defined in Equation (3), specify how person-hours spent on shared review and platform work are attributed to tasks. Record separately any work that the observations cannot attribute to a particular task. Define what counts as one agent instance and the categories of computing resource used when reporting its running time. Record those running times, token use and charges separately from person-hours. Combining these quantities requires an explicit economic conversion and changes the question from labour to cost.

Let T_i be elapsed time from the start of tracking to first acceptance. Starting this clock at s_i includes queueing and any reasoning and specification between s_i and the first commit. If the task is permanently abandoned before acceptance, put $T_i = \infty$. Abandonment is an outcome, whereas loss of observation is censoring. Record deployment separately whenever acceptance and deployment differ.

3.2. **Scoring tasks and checking retained output.** Calculate a nonnegative reference score S_i^0 for each task, recording which version of the scoring rule is used and which task information it takes as input. The score may count tasks or functional units, or use SCH weights. Retained output has the same unit as this score. Productivity divides that output by person-hours. The superscript 0 denotes the common reference: retain each task's score and use the same scoring rule and weights across comparison groups. A score expressed in person-hours needs calibration against independently measured labour. Specify whose labour is used for calibration, which types of work are counted, the quality requirements and the event at which labour recording ends. A predictor trained on imputed activity states learns to predict those states; its accuracy as a measure of person-hours still requires independent validation.

For a causal comparison with fixed task requirements, requirements recorded when tracking begins are preferable to final code differences as scoring inputs: AI can change the final code. A retrospective code score can still be reported, with its dependence on the implementation made explicit. A nonlinear scoring rule need not be additive, so splitting or squashing a change can alter its score. Score the same recorded task information once, or use a requirement-scoring rule whose component scores add to its total. A failed attempt, retry or revert must not generate another unit of retained output.

Choose an acceptance deadline at task age $\tau > 0$ and a follow-up duration $\Delta \geq 0$. State the criteria for judging what fraction of an accepted requirement remains satisfied. For accepted tasks, let $R_i(\Delta) \in [0, 1]$ be that fraction at time $s_i + T_i + \Delta$, judged using these criteria. Retained reference output is

$$Q_i(\tau, \Delta) = \begin{cases} S_i^0 R_i(\Delta), & T_i \leq \tau, \\ 0, & T_i > \tau. \end{cases} \tag{4}$$

Specify how the follow-up check distinguishes an intentional replacement from a defect-driven loss and how fractional scores are calculated. Surviving lines, successful tests and absence of recorded incidents each provide observations of quality, while leaving complete correctness unestablished. Measure agreement between reviewers and document which defects the tests and follow-up criteria can detect.

For a specified finite set of tasks with positive total labour, define

$$P_H(\tau, \Delta) = \frac{\sum_i Q_i(\tau, \Delta)}{\sum_i H_i(\tau + \Delta)}. \tag{5}$$

The denominator sums labour through the same task age $\tau + \Delta$ for every task in that set. It includes labour on rejected, unfinished and abandoned tasks, together with work done after acceptance that is attributable to the task. Early acceptance leaves more time for repair within this labour window than late acceptance. Stopping repair accounting exactly Δ after each acceptance would define a different denominator. State when labour recording ends and follow every task for $\tau + \Delta$ to measure both the retained output and labour defined here.

Tasks with zero human labour can contribute retained output. The productivity ratio is defined only when the tasks' total labour is positive. Report retained output and total person-hours separately.

3.3. **The observations and the three clocks.** Acceptance, retained output and labour use different timing rules: acceptance by the deadline τ, retention at task age $T_i + \Delta$, and labour accumulated through task age $\tau + \Delta$. A complete task history supplies the observations needed under each rule. Recording acceptance alone does not establish how much output survives or how much repair work follows. A record *point-identifies* realised labour when at least one time allocation satisfies the record and the model's constraints, and every such allocation gives the same total. Proposition 2.1 instead asks whether the probability distribution of observed events uniquely determines a population parameter. The next section derives the smallest and largest labour totals consistent with interval observations.

4. Attainable labour ranges from interval observations

Knowing how many person-hours occurred during an interval need not tell us when within that interval people worked. We seek every labour total that could have occurred within the specified target windows while satisfying the observations and the allocation constraint in (3). These totals form the *identified range*: for each total, at least one way of allocating people's time satisfies all the observations and constraints. No allocation satisfying them gives a total outside this range. This definition follows the general partial-identification perspective [13, Section 1]. The following proofs derive these totals from the stated observation bounds and allocation constraints.

4.1. **Using the person's capacity.** One person can contribute at most one person-hour in an elapsed hour. Knowing who supplied an interval total therefore limits how much of that labour could have occurred before or after a specified time. A total across several people must be compared with their combined available time.

Proposition 4.1 (Attainable contribution from one interval)**.** *Let $\ell < u$, $c \in [\ell, u]$ and $v \in [0, u - \ell]$. Among measurable allocations $a : [\ell, u] \to [0, 1]$ with $\int_\ell^u a(t)\,dt = v$, the attainable values of $x = \int_\ell^c a(t)\,dt$ are exactly*

$$[\max\{0, v - (u - c)\},\ \min\{v, c - \ell\}]\,. \tag{6}$$

If $v < 0$ or $v > u - \ell$, no such allocation exists.

Proof. Nonnegativity and capacity on the two pieces imply

$$0 \le x \le c - \ell, \qquad 0 \le v - x \le u - c.$$

Together, these inequalities give (6). Conversely, for any x in this interval, set $a = x/(c - \ell)$ before c and $a = (v - x)/(u - c)$ after c. Each value lies in $[0, 1]$ and gives the required integral. Omit a zero-length piece when c is an endpoint: the corresponding numerator is then zero. Finally, integrating $0 \le a \le 1$ gives the necessary condition $0 \le v \le u - \ell$. □

For one person, 3/2 person-hours in a two-hour interval imply between 1/2 and 1 person-hours in its first hour. Nonnegativity alone would give $[0, 3/2]$. In this example, the bounds do not determine the first-hour total or exactly when the person worked. If $v = u - \ell$, the total uses all the person's available time, so the labour before any time within it equals the elapsed time from its start. For an interior subinterval $[b, c] \subseteq [\ell, u]$, applying the same construction to that subinterval and the time outside it gives every possible labour total in the subinterval: $[\max\{0, v - ((u - \ell) - (c - b))\}, \min\{v, c - b\}]$.

Bounds calculated separately for each contribution can be added when each contribution is counted once. The summed lower bound is possible only if allocations achieving every component minimum can occur together; the same condition applies to the upper bound

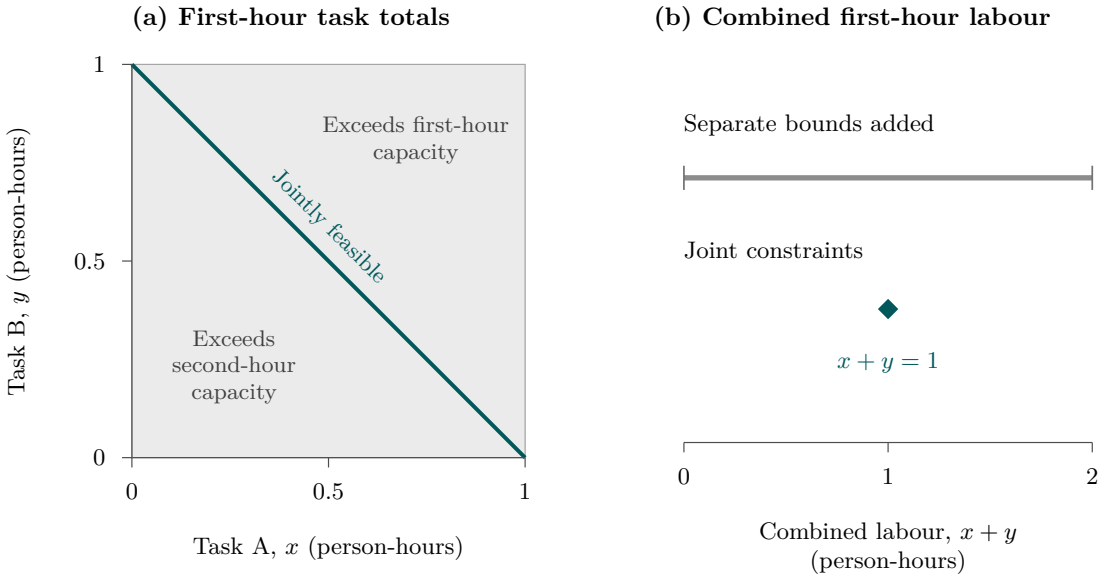


FIGURE 1. **Shared capacity determines the combined first-hour labour.** Constructed example: one person spends one person-hour on each of two tasks during two hours. In (a), x and y are the tasks' first-hour labour totals. The shaded square satisfies the separate bounds; only the diagonal satisfies the person's capacity in both hours. In (b), adding the separate bounds permits between zero and two person-hours, whereas the joint constraints require exactly one. The plotted points specify interval totals, not complete work schedules.

and component maxima. Disjoint intervals of one person, or intervals belonging to distinct people, can be combined when no other constraints link them. Overlapping observations or several tasks competing for the same person's time require a joint calculation. For example, one person spends one person-hour on each of two tasks during $[0, 2]$. Each task separately allows $[0, 1]$ person-hours before time 1. Together, the tasks require exactly one person-hour before time 1, because they use all the person's available time on $[0, 2]$. Adding the separately calculated bounds would give $[0, 2]$. Figure 1 plots the separately permitted and jointly feasible first-hour totals.

4.2. **Replacing allocation histories with interval totals.** To account for tasks sharing a person's time, call one person's work of one type on one task a stream. Fix finite sets of tasks $\mathcal{I}$, people $\mathcal{J}$ and types of work $\mathcal{R}$, a set of tasks to be studied $\mathcal{C} \subseteq \mathcal{I}$, and a bounded calendar-time domain $[b_0, b_K]$ with $b_0 < b_K$ containing every observation interval and target window. An admissible history consists of measurable $a_{ijr} \geq 0$ satisfying $\sum_{i,r} a_{ijr}(t) \leq 1$ almost everywhere for every person j. The list of people and the rules for attributing time to tasks determine whose labour and which work are included. Whether these records cover the intended work requires measurement evidence. Work outside $\mathcal{C}$ can be recorded separately when it uses the same people's time without contributing to the labour total for $\mathcal{C}$.

For observations $\nu = 1, \ldots, M$, specify an interval $[\ell_\nu, u_\nu]$ within this domain, a set $G_\nu \subseteq \mathcal{I} \times \mathcal{J} \times \mathcal{R}$ of attributed streams, and finite labour bounds $v_\nu^- \leq v_\nu^+$, in person-hours. Impose

$$v_\nu^- \leq \sum_{(i,j,r) \in G_\nu} \int_{\ell_\nu}^{u_\nu} a_{ijr}(t)\, dt \leq v_\nu^+. \tag{7}$$

Exact interval totals use $v_\nu^- = v_\nu^+$. Justified limits on rounding or measurement error may instead give different lower and upper bounds. A Horvitz–Thompson estimate varies with the selected sample; that sampling uncertainty requires a separate analysis before the estimate can justify either inequality.

Form the sorted distinct endpoints $b_0 < \cdots < b_K$ from the domain boundaries, every observation endpoint and every target boundary. Call the intervals between successive endpoints cells. For the denominator in (5), target boundaries include the start of tracking s_i and the end of the labour observation window $c_i = s_i + \tau + \Delta$ for every task in $\mathcal{C}$. Translate time since tracking began into calendar time before partitioning, so that simultaneous work by the same person shares one capacity constraint. Keep cells outside a target window whenever an observation spans them: some of the observed labour may have occurred outside the window. Put $d_k = b_k - b_{k-1} > 0$ and let x_{ijrk} denote the labour allocated to stream (i, j, r) in cell k.

Proposition 4.2 (Exact cell reduction and attainable labour range)**.** *Under the preceding model, define the finite-dimensional feasible set $\mathcal{P}$ by*

$$x_{ijrk} \geq 0, \qquad \sum_{i,r} x_{ijrk} \leq d_k \qquad \text{for every } j, k, \tag{8}$$

$$v_\nu^- \leq \sum_{(i,j,r)\in G_\nu} \sum_{k:\,[b_{k-1},b_k]\subseteq[\ell_\nu,u_\nu]} x_{ijrk} \leq v_\nu^+ \qquad \text{for every } \nu. \tag{9}$$

Fix real weights w_{ijrk} and define the target for a continuous-time allocation history by multiplying each cell integral by w_{ijrk} and summing. The attainable target values over histories satisfying the capacity and observation constraints equal $h(\mathcal{P})$, where $h(x) = \sum_{i,j,r,k} w_{ijrk} x_{ijrk}$. In particular, the denominator in (5) uses $w_{ijrk} = 1$ exactly when $i \in \mathcal{C}$ and $[b_{k-1}, b_k] \subseteq [s_i, c_i]$, and zero otherwise. If $\mathcal{P}$ is nonempty, its target range is the closed interval

$$[H_{\min}, H_{\max}] = \left[\min_{x\in\mathcal{P}} h(x),\ \max_{x\in\mathcal{P}} h(x)\right]. \tag{10}$$

Both endpoints and every value between them are attained, even if every $a_{ijr}(t)$ is restricted to $\{0, 1\}$, so that each person does at most one type of work on one task at a time. If $\mathcal{P}$ is empty, the observations and capacity assumptions are inconsistent.

Proof. Integrating any such history over each cell gives nonnegative cell labour totals. Integrating each person's allocation limit gives (8). Splitting each observation at the cell boundaries gives (9). The weighted target total is unchanged because its weights are constant on cells.

Conversely, fix $x \in \mathcal{P}$. For each person and cell, order the task–type-of-work pairs and divide the cell into consecutive blocks whose lengths are their respective x_{ijrk}. The capacity constraint ensures that these blocks fit inside the cell. Use half-open blocks to avoid overlap at their boundaries. During each block, set the corresponding allocation to one and all others to zero; leave any remaining time idle. Omit zero-length blocks and give the final domain endpoint the same allocation values as the last block. These measurable allocations reproduce every cell labour total, so all observation bounds remain satisfied and the target value is unchanged. Each person works on at most one task and type of work at a time, with finitely many changes. Thus even these restricted allocation functions give exactly the same possible target values as the finite feasible set.

The constraints define a closed convex set. Each coordinate is between zero and d_k, so a nonempty $\mathcal{P}$ is compact and a linear functional attains its extrema. Convex combinations of minimising and maximising cell vectors attain every intermediate value. Reconstructing each such vector by consecutive blocks supplies an allocation with values in $\{0, 1\}$ for each intermediate total. □

The minimum and maximum target totals can be computed by linear programmes, each with $|\mathcal{I}||\mathcal{J}||\mathcal{R}|K$ variables, $|\mathcal{J}|K$ shared-capacity constraints and $2M$ observation inequalities, in addition to nonnegativity. A known period of unavailability can be included by adding its boundaries to the partition and setting that person's cell labour totals to zero within it. The reconstruction uses at most $|\mathcal{I}||\mathcal{R}| + 1$ constant blocks per person in each cell, including idle time. Fractional cell totals therefore need not describe simultaneous fractional attention. Work must be allowed to start, stop and change type or task at arbitrary times within a cell. Minimum session lengths, restrictions on interruption, smoothness constraints and switching costs require a separate analysis.

4.3. **Checking the bounds and allowing observation error.** Collect the cell totals and target weights into vectors x and w, and write the capacity and observation inequalities as $Ax \le b$, with $x \ge 0$ kept separate. A lower observation bound is included by reversing its sign. Nonnegative multipliers λ and μ satisfying

$$A^{\mathsf{T}}\lambda \ge w, \qquad A^{\mathsf{T}}\mu \ge -w \quad \Longrightarrow \quad -\mu^{\mathsf{T}}b \le h(x) \le \lambda^{\mathsf{T}}b \qquad (x \in \mathcal{P}) \tag{11}$$

give bounds by weighted sums of the original inequalities: $w^{\mathsf{T}}x \le \lambda^{\mathsf{T}}Ax \le \lambda^{\mathsf{T}}b$, and likewise for $-w$. For a nonempty $\mathcal{P}$, linear-programming strong duality ensures that suitable multipliers give exactly $H_{\min}$ and $H_{\max}$ [4, Sections 5.2–5.3]. An allocation attaining an endpoint and multipliers giving the same bound together certify that endpoint. Rational certificates can be checked by exact arithmetic without repeating the optimisation. A numerical solver's success flag alone does not supply this check.

Increasing an upper observation bound or decreasing a lower one increases the corresponding entry of b. If these changes give $b + \varepsilon$, with $\varepsilon \ge 0$ and zero entries for unchanged constraints, the same multipliers bound every newly permitted target total by

$$[-\mu^{\mathsf{T}}(b + \varepsilon),\ \lambda^{\mathsf{T}}(b + \varepsilon)]. \tag{12}$$

This follows by the same weighted-inequality argument; it is a sensitivity bound of the kind discussed in [4, Section 5.6]. Reoptimising may give a smaller interval. The error limits still need justification from the observations.

Sampling requires an additional coverage argument. Suppose random lower and upper bounds contain all true observation totals simultaneously with probability at least $1 - \alpha$, where $0 < \alpha < 1$. If the true history satisfies the allocation constraints, its cell vector belongs to the resulting random feasible set on that event. Optimising then gives an interval containing the true target total with probability at least $1 - \alpha$. Count an empty feasible set as a failure to cover. Separate coverage of $1 - \alpha$ for each observation does not establish the same simultaneous coverage; marginal error probabilities whose sum is at most α suffice by the union bound. Constructing these bounds requires an argument for the actual sampling and measurement process. The resulting interval has a sampling-coverage guarantee, rather than the exact-data interpretation of (10).

4.4. **From labour bounds to productivity.** For the denominator in (5), let $Q = \sum_{i \in \mathcal{C}} Q_i(\tau, \Delta)$ be retained output known exactly. With $Q \ge 0$ and $H_{\min} > 0$, dividing this output by the largest and smallest possible labour totals gives the identified productivity interval $[Q/H_{\max}, Q/H_{\min}]$. Justified quality bounds $0 \le Q_- \le Q \le Q_+$ give an interval $[Q_-/H_{\max}, Q_+/H_{\min}]$ containing every feasible output-to-labour ratio. It is the exact range only when every ratio in it can be produced by an output total and labour total that are jointly permitted. It is sufficient for every pair in $[Q_-, Q_+] \times [H_{\min}, H_{\max}]$ to be permitted. A lower labour bound of zero cannot be used as a divisor. The acceptance times and the recorded fractions of requirements still met in (4) remain fixed throughout the allocation optimisation.

5. Estimating labour from sampled allocations

The preceding bounds assume exact interval labour totals or justified lower and upper bounds on them. A probability sample instead gives an estimate that varies with the selected person-intervals. Treating that estimate as an exact interval total can rule out the true work allocation because of sampling error. The difference estimator below corrects telemetry predictions using sampled labour. Under explicit sampling and measurement assumptions, its average over repeated samples equals the true labour total.

Activity sampling estimates labour from selected person-intervals. The sampling frame—the complete list of intervals from which the sample is drawn—must cover all target work, including off-keyboard reasoning, review and coordination. Short diaries and task records can show how much of each selected interval a person spent on the target work. For a subsample, state who checks the recorded allocations, what evidence they use and how they remain independent of the original recording. Report disagreement. Nonresponse, inaccurate recall and work outside the frame require separate investigation.

5.1. **Predictions calculated before sampling.**

The difference estimator starts with a fixed prediction of total labour and corrects it using a probability sample of allocation observations. For each sampled person-interval, subtract predicted labour from observed labour and divide by the interval's inclusion probability. Under the conditions below, the sum of these weighted residuals averages over repeated samples to the true total minus the prediction total. Denis and Haddou give this fixed-prediction estimator and its variance under the sampling design explicitly [7, Section 2.3, equations (2)–(4)]. Prediction-powered inference also identifies its estimator of a mean as a difference estimator and considers finite-population extensions [1, Section 1.4 and Appendix B of the accompanying preprint].

Partition each person's target observation period into non-overlapping intervals. Let $U = \{1, \dots, M\}$, $M \geq 1$, be the complete, fixed frame of these person-intervals. For interval m, let $d_m > 0$ be its length in hours and $a_m \in [0, 1]$ its average allocation to the target work. Its labour is $y_m = d_m a_m$ person-hours, and the population total is $H = \sum_U y_m$. For every frame unit, retain a prediction f_m in person-hours. A raw SCH value requires a stated conversion to labour units before it can serve as f_m. Each unit's prediction must remain the same whichever sample is drawn. Training on an independent earlier population can satisfy this condition: hold all predictions fixed, then sample allocations to correct the predicted total.

Proposition 5.1 (Fixed-prediction difference estimator)**.** *For a probability sample $s \subseteq U$ with known first-order inclusion probabilities $\pi_m > 0$, set $r_m = y_m - f_m$ and*

$$\widehat{H}_{\text{diff}} = \sum_{m \in U} f_m + \sum_{m \in s} \frac{y_m - f_m}{\pi_m}. \tag{13}$$

If y_m and f_m are fixed under the design and every sampled labour value is observed accurately, then $\mathbb{E}_s[\widehat{H}_{\text{diff}}] = H$, where $\mathbb{E}_s$ averages over the sampling design.

Proof. Write $I_m = \mathbf{1}\{m \in s\}$. Linearity and $\mathbb{E}_s[I_m] = \pi_m$ give

$$\mathbb{E}_s[\widehat{H}_{\text{diff}}] = \sum_U f_m + \sum_U \frac{\mathbb{E}_s[I_m]}{\pi_m}(y_m - f_m) = \sum_U y_m.$$

□

The residual correction makes the estimate design-unbiased even when the fixed prediction total differs from H. Taking $f_m = 0$ gives the Horvitz–Thompson (HT) estimator [9],

$$\widehat{H}_{\text{HT}} = \sum_{m \in s} \frac{d_m a_m}{\pi_m}. \tag{14}$$

For every frame interval, record one of three states: selected with an observed allocation (including zero), selected with no recorded allocation value (nonresponse), or unselected. The proof concerns true allocation values. Unbiasedness in the presence of measurement error requires additional assumptions on that error.

5.2. **Precision depends on residuals and the sampling design.** Let $\pi_{mk} = \mathbb{E}_s[I_m I_k]$ be the probability that both frame units are selected, including $\pi_{mm} = \pi_m$. The exact variance over repeated samples is

$$\operatorname{Var}_s(\widehat{H}_{\text{diff}}) = \sum_{m,k \in U} \frac{\pi_{mk} - \pi_m \pi_k}{\pi_m \pi_k} r_m r_k. \tag{15}$$

Expanding the variance of $\sum_U I_m r_m / \pi_m$ gives this expression. The calculation depends on how often pairs of units are selected together, through the covariances of their inclusion indicators. The variance formula in (15) also uses residuals for unsampled units, which are generally unknown. To estimate it, retain the actual sampling design and satisfy the chosen variance estimator's conditions on joint inclusion probabilities.

For simple random sampling without replacement from $M \geq 2$ units, let the fixed sample size satisfy $1 \leq n \leq M$. Every size-n subset has the same probability and $\pi_m = n/M$. Write

$$S_r^2 = \frac{1}{M-1} \sum_U (r_m - \bar{r}_U)^2, \qquad S_y^2 = \frac{1}{M-1} \sum_U (y_m - \bar{y}_U)^2.$$

The inclusion covariances in (15) give

$$\operatorname{Var}_s(\widehat{H}_{\text{diff}}) = \frac{M^2}{n} \left(1 - \frac{n}{M}\right) S_r^2, \qquad \operatorname{Var}_s(\widehat{H}_{\text{HT}}) = \frac{M^2}{n} \left(1 - \frac{n}{M}\right) S_y^2. \tag{16}$$

For $2 \leq n \leq M$, an unbiased estimator of the design variance is

$$\widehat{V}_{\text{diff}} = \frac{M^2}{n} \left(1 - \frac{n}{M}\right) s_r^2, \qquad s_r^2 = \frac{1}{n-1} \sum_{m \in s} (r_m - \bar{r}_s)^2. \tag{17}$$

Indeed, $s_r^2 = [n(n-1)]^{-1} \sum_{m<k,\, m,k \in s} (r_m - r_k)^2$. Each pair has inclusion probability $n(n-1)/[M(M-1)]$, so $\mathbb{E}_s[s_r^2] = S_r^2$. Selecting one unit from a population containing more than one unit does not supply this variance estimate. Measuring every unit gives zero sampling variance, including in a single-unit population. Measurement error remains a separate question.

For $n < M$, the difference estimator has lower sampling variance than HT exactly when $S_r^2 < S_y^2$: the residuals must have lower population variance than the person-interval labour totals. Consider $M = 4$, $n = 2$, and $y = (0, 2, 4, 6)$. The true total is 12, and HT variance is $80/3$. Using $f = (1, 3, 3, 5)$ for every sample yields variance $16/3$; using $f = (6, 4, 2, 0)$ yields $320/3$. Enumerating all six samples reproduces these variances and confirms that each estimator averages to the true total. It also verifies that (17) averages to each corresponding variance. The second predictor gives four times the HT variance. The difference estimator remains design-unbiased with either predictor. These finite checks establish the calculations for these two predictors. A workplace study must determine whether its code-derived predictions give a residual total with lower sampling variance than the labour estimate without predictions.

5.3. **Failure cases and implementation checks.** The proof above does not cover predictions fitted to the sample used for correction. For $M = 2$, $n = 1$ and $y = (0, 2)$, suppose a fitted rule memorises the selected outcome and predicts zero for the other unit. Its observed residual is always zero. The difference estimate is nevertheless 0 or 2, each with probability $1/2$, giving expectation 1 rather than $H = 2$. A method trained on the sampled units needs a separate unbiasedness argument under the actual finite-population sampling design.

The estimate can also be negative even though true labour is nonnegative. With $y = (0, 0)$, $f = (10, 0)$ and one uniformly selected unit, (13) is -10 or 10 with equal probability. Clipping

at zero changes its expectation from 0 to 5. Preserve the signed estimate when reporting this estimator and investigate an implausible result. Dividing retained output by an unbiased labour estimate also need not yield an unbiased productivity estimate.

The accompanying implementation reports estimates with their calculated sign, preserves observed zeros, and records nonresponse separately from unselected units. It rejects inputs with missing required information, selected intervals without allocation measurements, and inconsistent values. For a supplied general sampling design it reports point estimates. For simple random sampling it also reports variance and standard error under the conditions above. These checks establish whether supplied records satisfy the input rules. Establishing that the frame covers the work, allocations are measured accurately and units were sampled by the stated method requires evidence from collection.

Exhaustive finite tests check the examples above, input validation, complete censuses and single-unit populations. A confidence interval requires a separate argument that its stated coverage holds under the sampling design.

6. Validating the measurements

An empirical study should first test whether the proposed observations measure person-hours accurately and cover all the included work. Before collection, state which tasks and types of work are included and how person-hours are attributed to tasks. For that population, specify the sampling frame, reference scoring rule, acceptance deadline and criteria for checking retained output. Measure disagreement over recorded allocations. Also record nonresponse, follow-up completeness and the time spent collecting records. Report these measurements by comparison group, type of work and task category, stating how tasks are categorised. Use these observations to choose how many person-intervals to sample and how precisely a later study should estimate differences between the groups.

For interval bounds, use independently checked totals with justified limits on measurement error. For target windows chosen before observing outcomes, compare the sums of separately calculated lower and upper labour bounds with the joint minimum and maximum. This shows whether accounting jointly for each person's available time raises the minimum or lowers the maximum. For sampled labour, compare the variance over repeated samples with and without predictions calculated before sampling and kept unchanged whichever sample is drawn. Retain the inclusion probabilities and joint sampling-design information needed to calculate uncertainty.

6.1. **Comparing predictions with sampled labour.**

A sampled labour estimate can differ from true labour, so its difference from a prediction includes sampling error. Compare telemetry predictions with labour recorded for teams or periods excluded when fitting the prediction rule. Keep predictions unchanged whichever validation sample is drawn, allow for sampling error, and report comparisons by study group, type of work and task category. State how tasks are categorised. Correlation between group averages does not establish whether predictions systematically overstate or understate labour, or how far individual predictions are from true labour. The following calculations separate prediction error against true labour from sampling variance.

Treat each task's labour H in a finite validation population as fixed. Let f be its prediction, calculated before validation sampling and unchanged whichever sample is drawn, and suppose $\mathbb{E}_s[\widehat{H}] = H$ with finite design variance. Then

$$\mathbb{E}_s[f - \widehat{H}] = f - H,$$
$$\mathbb{E}_s[(f - \widehat{H})^2] = (f - H)^2 + \mathrm{Var}_s(\widehat{H}).$$

Expand the square: the cross term vanishes because $\mathbb{E}_s[\widehat{H} - H] = 0$. These identities hold for each task under its stated sampling design. Aggregate uncertainty additionally depends on covariances between tasks' estimates.

For a second prediction g calculated before the same validation sample and unchanged whichever sample is drawn, subtraction cancels the common sampling variance:

$$\mathbb{E}_s[(f - \widehat{H})^2 - (g - \widehat{H})^2] = (f - H)^2 - (g - H)^2.$$

Over repeated validation samples, the average difference between the two squared prediction errors equals their difference calculated against true labour. Estimating either predictor's squared error against true labour, or quantifying sampling uncertainty in their comparison, requires further sampling-design information. Absolute error requires a separate analysis.

For example, eight one-hour person-intervals have allocations $(1, 1, 1, 1, 0, 0, 0, 0)$. Sample one uniformly, with inclusion probability $1/8$. True labour is $H = 4$, whereas its HT estimate is 0 or 8, each with probability $1/2$. A perfect prediction $f = 4$ has apparent absolute error 4 against the sampled estimate. The interval $[3.5, 4.5]$ always contains true labour but never contains the sampled estimate. The signed difference between prediction and sampled estimate remains unbiased for the prediction's error against true labour. Absolute error and interval coverage calculated against $\widehat{H}$ measure agreement with the sampled estimate, including its sampling error.

To examine accuracy against true labour, select a validation subset of tasks and record allocations for every person-interval in each task's target window. State who checks those records, what evidence they use to resolve discrepancies and how they remain independent of the original recording. Report remaining measurement error. Alternatively, specify a measurement model under which the observations uniquely determine the chosen measure of prediction accuracy. Retain inclusion probabilities and joint sampling-design information for uncertainty calculations.

Validation should also test whether task splitting, code-change squashing or changes to the task information supplied to the scoring rule alter the reference score. Report reviewer disagreement and follow-up completeness alongside prediction error. Recalculate comparisons between study groups using plausible amounts of missing labour and disputed retained-output scores. If these changes reverse which group has the better outcome, choosing between the AI tools and working practices requires resolving or accepting the measurement uncertainty.

6.2. **Conditions for comparing AI tools and working practices.** A subsequent causal study must define whether it concerns a fixed set of tasks or an organisation's incoming work that meets stated conditions for inclusion. AI adoption can change which work is requested or undertaken. For a fixed-workload comparison, set the conditions for including tasks before allocating tasks, people or teams to comparison groups, and keep those conditions unchanged. State how work done before group allocation is counted. Record postponed, rejected and abandoned requests. An organisation-wide adoption comparison instead needs a common calendar window and records of incoming requests and work in progress. State which population each comparison concerns and how its observations are weighted.

The AI tools and working practices specified for one team can affect another team through shared reviewers, limited computing resources for agents or reusable components. Account for these effects when forming comparison groups and analysing outcomes. Analyse outcomes by the original allocation to comparison groups, recording actual tool use separately. Calculate uncertainty using the same teams, people or other units that were allocated to those groups, with additional allowance for sampling labour and checking quality in subsets. Recording more commits does not increase the number of independently allocated units.

Use the same criteria and follow-up procedure for checking retained output across comparison groups. Have two reviewers check a subset independently to measure disagreement between them. Record any loss of reviewer blinding—reviewers learning which group a task belongs to—and the timing of feedback, since the check can trigger repair. Count that repair within the stated labour window. Estimating retained output for all tasks from a checked subset requires known selection probabilities and assumptions addressing any effect of the

check on subsequent work. Report severe defects separately because a retained fraction cannot describe every security or reliability consequence.

Complete follow-up through $\tau + \Delta$ as required by Section 3. Distinguish permanent abandonment from loss of observation. If follow-up is incomplete, state the assumptions or bounds used to handle missing outcomes. Report labour, acceptance and retained output together, since reduced labour may accompany more abandonment. Report retained output and positive total labour alongside their productivity ratio, and account for uncertainty in both quantities and their dependence. Prespecify practically useful improvement thresholds and any maximum acceptable reduction in quality (the noninferiority margin). Failure to detect a quality reduction does not establish that the loss is within that limit.

7. Scope and next observations

Different distributions of human activity can produce identical commit distributions. Interval totals or justified bounds, together with each person's available time, restrict how much work could have occurred during a specified period. Splitting time at every observation endpoint and target-window boundary gives a finite optimisation problem with exactly the same possible labour totals. Adding separately calculated interval bounds can include totals that would require someone to work beyond their available time. The summed lower and upper bounds are exact only if work allocations achieving the respective component bounds can occur together.

Probability sampling permits design-unbiased estimation when the frame covers the target work, inclusion probabilities are known and positive, and sampled allocations are measured accurately. A fixed telemetry predictor improves precision only when estimating its total residual has lower sampling variance than estimating labour without predictions. Diaries and task records can supply the observations. Whether they capture reasoning and coordination and accurately record how people divide their time requires empirical evidence. The unbiasedness and precision results depend on the stated measurement and sampling assumptions. Establishing a causal benefit from AI adoption requires a further comparison of AI tools and working practices.

The next empirical test is whether task records can account for person-hours spent on all included types of work and independently checked retained output through the required follow-up. Disagreement over recorded allocations, missing records and the time needed to collect them will help determine how much measurement is practicable before a causal study is designed. That study must also choose whether to compare a fixed workload or organisation-wide adoption that can change incoming requirements. The results specify which labour totals are consistent with the observations, when sampling gives an unbiased estimate and when fixed predictions reduce its variance.

Appendix A. A shortest-path solution for one allocation stream

For one person's work of one type on one task, let $0 \le a \le 1$ be the allocation function. The interval observations of Section 4 can be represented by cumulative labour at their endpoints. Using the sorted endpoints and cell lengths defined there, set $z_k = \int_{b_0}^{b_k} a(t)\,dt$. Write an inequality $z_q - z_p \le w$ as a directed edge $p \to q$ of weight w. Adjacent endpoints receive edges $k-1 \to k$ of weight d_k and $k \to k-1$ of weight zero. Observation bounds on $[b_p, b_q]$ add edges $p \to q$ of weight v^+ and $q \to p$ of weight $-v^-$. An exact observed labour total v therefore gives edge weights v and $-v$. This is the standard difference-constraints representation [10, Slides 6–11].

Proposition A.1 (Attainable cumulative bounds from graph distances)**.** *The scalar observations admit an allocation if and only if this graph has no directed cycle of negative total weight. When it is feasible, let $D(p, q)$ be the shortest-path weight from p to q. For any*

endpoint pair $p \le q$, the identified range of $\int_{b_p}^{b_q} a(t)\,dt$ is

$$[-D(q,p),\ D(p,q)]. \tag{18}$$

Proof. Summing edge inequalities around a cycle gives 0 at the left, so a negative cycle is inconsistent. Conversely, adjacent edges make every vertex reachable from every other. In the absence of negative cycles, all shortest-path weights are finite and $D(r,r) = 0$. For any starting vertex r, the values $z_k = D(r,k)$ satisfy each edge inequality because $D(r,v) \le D(r,u) + w$ for an edge $u \to v$ of weight w. Subtracting z_0 makes initial cumulative labour zero without changing differences. The inequalities for adjacent endpoints give cell labour totals in $[0, d_k]$. Allocations constant within each cell can produce these totals and satisfy the observations.

Summing inequalities along paths yields $z_q - z_p \le D(p,q)$ and $z_q - z_p \ge -D(q,p)$. Using p as the starting vertex gives an allocation achieving the upper bound; using q gives an allocation achieving the lower bound. Convex combinations of these allocations give every labour total between those bounds. □

The reference checker uses exact rational arithmetic and an all-pairs shortest-path calculation. Given indexed, sorted endpoints, graph construction and this calculation use $O((K+1)^3 + M)$ arithmetic operations and $O((K+1)^2 + M)$ storage. Bit costs depend on the encoded rational inputs. The checker compares the bounds and the allocations attaining them against an independent enumeration of finite allocations, including overlapping interval observations and inconsistent inputs. These constructed examples check the calculation against the proved result. Empirical measurements are still needed to validate the observation process. Several task streams sharing a person's capacity require the joint formulation in Proposition 4.2, because the scalar graph represents only one allocation.

References

1. Anastasios N. Angelopoulos, Stephen Bates, Clara Fannjiang, Michael I. Jordan, and Tijana Zrnic, *Prediction-powered inference*, Science **382** (2023), no. 6671, 669–674, Author manuscript: arXiv:2301.09633v4. https://doi.org/10.1126/science.adi6000.
2. Joel Becker, Nate Rush, Elizabeth Barnes, and David Rein, *Measuring the impact of early-2025 AI on experienced open-source developer productivity*, 2025, arXiv:2507.09089v2. https://arxiv.org/abs/2507.09089v2.
3. Joel Becker, Nate Rush, Tom Cunningham, David Rein, and Khalid Mahamud, *We are changing our developer productivity experiment design*, METR research update, February 2026, 24 February 2026. https://metr.org/blog/2026-02-24-uplift-update/.
4. Stephen Boyd and Lieven Vandenberghe, *Convex optimization*, Cambridge University Press, 2004, Sections 5.2–5.3 and 5.6. https://web.stanford.edu/~boyd/cvxbook/bv_cvxbook.pdf.
5. Kevin Zheyuan Cui, Mert Demirer, Sonia Jaffe, Leon Musolff, Sida Peng, and Tobias Salz, *The effects of generative AI on high-skilled work: Evidence from three field experiments with software developers*, June 2025, Author manuscript; Section 4.1 and Table 3. https://www.mertdemirer.com/Papers/Demirer_AI_productivity.pdf.
6. ———, *The effects of generative AI on high-skilled work: Evidence from three field experiments with software developers*, Management Science (2026), Published online 27 February 2026. https://doi.org/10.1287/mnsc.2025.00535.
7. Nicholas Denis and Mohammed Haddou, *Prediction-powered estimation: Unbiased model-assisted estimation*, Journal of Official Statistics **42** (2026), no. 3, 371–394, https://doi.org/10.1177/0282423X261451320.
8. Nicole Forsgren, Margaret-Anne Storey, Chandra Maddila, Thomas Zimmermann, Brian Houck, and Jenna Butler, *The SPACE of developer productivity: There's more to it than you think*, ACM Queue **19** (2021), no. 1, 20–48, https://doi.org/10.1145/3454122.3454124.

9. D. G. Horvitz and D. J. Thompson, *A generalization of sampling without replacement from a finite universe*, Journal of the American Statistical Association **47** (1952), no. 260, 663–685, https://doi.org/10.1080/01621459.1952.10483446.
10. Piotr Indyk, *Lecture 17: Shortest paths IV: Applications*, 6.006 Introduction to Algorithms, Massachusetts Institute of Technology, 2011, Lecture slides, 7 April; difference constraints on slides 6–11. https://courses.csail.mit.edu/6.006/spring11/lectures/lec17.pdf.
11. Hussein Mozannar, Gagan Bansal, Adam Fourney, and Eric Horvitz, *Reading between the lines: Modeling user behavior and costs in AI-assisted programming*, Proceedings of the CHI Conference on Human Factors in Computing Systems, Association for Computing Machinery, 2024, Author manuscript: arXiv:2210.14306v5. https://arxiv.org/abs/2210.14306v5, pp. 142:1–142:16.
12. Sida Peng, Eirini Kalliamvakou, Peter Cihon, and Mert Demirer, *The impact of AI on developer productivity: Evidence from GitHub Copilot*, 2023, arXiv:2302.06590v1. https://arxiv.org/abs/2302.06590v1.
13. Alexander Torgovitsky, *Partial identification by extending subdistributions*, Quantitative Economics **10** (2019), no. 1, 105–144, https://doi.org/10.3982/QE634.
14. Ian Wright and Albert Ziegler, *The standard coder: a machine learning approach to measuring the effort required to produce source code change*, 2019, arXiv:1903.02436v1. https://arxiv.org/abs/1903.02436v1.